



Desarrollo de Software en Sistemas Distribuidos

Trabajo Práctico Integrador 2025

Entrega 3

Grupo 18

Integrantes

- Fermin Pierini - 21044/1
- Patricio Benjamín Serres - 21537/9
- Pedro Spadari - 21586/8

Integrantes.....	1
API - Cloud.....	1
Swagger API Cloud.....	1
Grooming de las actividades de la etapa 3.....	7
Estimaciones etapa 4.....	8

API - Cloud

URL base de la API en el cloud (Render): <https://tp-dssd-cloudapi.onrender.com>

Swagger API Cloud

URL: <https://tp-dssd-cloudapi.onrender.com/swagger-ui/index.html>



Servicios / Endpoints:

Autenticación:

Request:

POST - /auth/login

- ***Caso 1 - Éxito:***

Request Body:

```
{"username": "admin", "password": "12341234"}
```

Response:

```
{  
  "data": {  
    "token":  
    "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJmZXJtaW4iLCJpYXQiOiE3NjEyMjk  
yNjQsImV4cCI6MTc2MTIzMjg2NH0.ZOsKIAO10RyzRo2HnBHAP0umu8PM2TdV  
lpsGehcp2mM",  
    "tokenType": "Bearer",  
    "expiresIn": 0  
  }  
}
```

- ***Caso 2 - Error por Usuario y/o contraseña inválidos:***

Request Body:

```
{"username": "juanperez", "password": "12345abcd"}
```

Response:

```
{  
  "data": null,  
  "token": null,  
  "ok": false,  
  "message": "Invalid credentials",  
  "status": "UNAUTHORIZED"  
}
```

IMPORTANTE:

Para el resto de los endpoints listados a continuación, debe agregarse:

HEADER:

Authorization: Bearer <token>

El "<token>" debe ser el devuelto en el campo "token" del response del endpoint de login.



Por ej.: *Authorization: Bearer*

eyJhbGciOiJIUzI1NiJ9.eyJzdWwiOiJmZXJtaW4iLCJpYXQiOiE3NjEyMjkyNjQsImV4cCI6MTc2MTIzMjg2NH0.ZOsKIAO10RyzRo2HnBHAP0umu8PM2TdVlpsGehcp2mM

Pedidos:

Crear pedido de colaboración:

Request:

- *Caso 1 - Pedido cargado exitosamente:*
POST - /proyecto/1/etapa/1/pedidos

Request Body:

```
{
  "tipo": "MATERIALES",
  "descripcion": "Necesito ladrillos",
  "cantidad": 16,
  "unidades": "unidades"
}
```

- *Caso 2 - Error en la carga del pedido por proyecto inexistente:*
En este caso, lo que difiere es la url:

POST - /proyecto/1000/etapa/3/pedidos

Request Body:

```
{
  "tipo": "MANO_DE_OBRA",
  "descripcion": "Pintura interior aula",
  "cantidad": 16,
  "unidades": "horas"
}
```

Response:

```
{
  "data": null,
  "token": null,
  "ok": false,
  "message": "Proyecto no encontrado: 1000",
  "status": "NOT_FOUND"
}
```

- *Caso 3 - Error en la carga del pedido por etapa inexistente:*
En este caso difiere la url:

POST - /proyecto/1/etapa/3000/pedidos

Request Body:

```
{
  "tipo": "MANO_DE_OBRA",
  "descripcion": "Pintura interior aula",
  "cantidad": 16,

```



```
"unidades": "horas"
}
```

Response:

```
{
  "data": null,
  "token": null,
  "ok": false,
  "message": "Etapa no encontrada: 3000",
  "status": "NOT_FOUND"
}
```

Obtener pedidos de colaboración:

Request:

GET - /proyecto/1/pedidos

Response:

```
{
  "data": [
    {
      "id": 1,
      "proyectoId": 1,
      "etapaId": 3,
      "tipo": "MATERIALES",
      "descripcion": "Pintura interior aula",
      "cantidad": 16.00,
      "unidades": "litros"
    }
  ],
  "token": null,
  "ok": true,
  "message": "",
  "status": "OK"
}
```

Compromisos:

ONG - Comprometerse con pedido de colaboración:

Request:

- *Caso 1 - compromiso establecido exitosamente:*

POST - /compromisos/pedido/11/comprometer

Request Body:

```
{
  "ongId": 6,
```



- ```
"detalle": "La ONG aportará ladrillos",
"aporteMaterial": "10 ladrillos",
"aporteMonetario": null,
"fechaInicio": "2025-10-20",
"fechaFin": "2025-11-20"
}
```
- **Caso 2 - Error por id de pedido inexistente:**  
En este caso difiere la url:  
**POST** - /compromisos/pedido/100/comprometer  
**Request Body:**

```
{
 "ongId": 1,
 "detalle": "La ONG aportará 10 cajas de alimentos no
perecederos",
 "aporteMaterial": "10 cajas de alimentos",
 "aporteMonetario": null,
 "fechaInicio": "2025-10-20",
 "fechaFin": "2025-11-20"
}
```

  
**Response:**

```
{
 "data": null,
 "token": null,
 "ok": false,
 "message": "Pedido de colaboración no encontrado: 100",
 "status": "NOT_FOUND"
}
```
  - **Caso 3 - Error por ONG originante no se puede comprometer:**  
**Se modifica la onglId.**  
**POST** - /compromisos/pedido/11/comprometer  
**Request Body:**

```
{
 "ongId": 5,
 "detalle": "La ONG aportará ladrillos",
 "aporteMaterial": "10 ladrillos",
 "aporteMonetario": null,
 "fechaInicio": "2025-10-20",
 "fechaFin": "2025-11-20"
}
```

  
**Response:**

```
{
 "data": null,
 "ok": false,
 "message": "La ONG no puede comprometerse con un pedido
de un proyecto del que es originante",
 "status": "BAD_REQUEST"
}
```



}

### ONG - Cumplir compromiso:

#### Request:

- *Caso 1 - compromiso cumplido exitosamente:*  
**PUT** - /compromisos/30/ong/5/cumplir

Para probar el compromiso hecho en el endpoint anterior (se marca automáticamente como aceptado para poder testearlo):

**PUT** - /compromisos/{id}/ong/6/cumplir

{id} es el que devuelve la respuesta del endpoint anterior en un caso satisfactorio.

- *Caso 2 - Error al cumplir compromiso por id de compromiso invalido:*  
En este caso difiere la url:  
**PUT** - /compromisos/100/ong/6/cumplir

#### Response:

```
{
 "data": null,
 "token": null,
 "ok": false,
 "message": "Compromiso no encontrado: 100",
 "status": "NOT_FOUND"
}
```

- *Caso 3 - Error al cumplir compromiso por id de ong invalido:*  
En este caso difiere la url:  
**PUT** - /compromisos/31/ong/5/cumplir  
Es el compromiso de otra ONG

#### Response:

```
{
 "data": null,
 "token": null,
 "ok": false,
 "message": "La ONG no tiene permisos para modificar este compromiso",
 "status": "BAD_REQUEST"
}
```

- *Caso 4 - Error al cumplir compromiso por id de ong invalido:*  
En este caso difiere la url:  
**PUT** - /compromisos/31/ong/100/cumplir  
Es el compromiso de otra ONG

#### Response:

```
{
 "data": null,
```



```
"token": null,
"ok": false,
"message": "Organización no encontrada: 100",
"status": "NOT_FOUND"
}
```

- *Caso 5 - Error por estado invalido:*

En este caso difiere la url:

**PUT** - /compromisos/32/ong/5/cumplir

Es el compromiso de otra ONG

*Response:*

```
{
 "data": null,
 "ok": false,
 "message": "El compromiso debe estar en estado 'ACEPTADO'
para marcarlo como cumplido",
 "status": "BAD_REQUEST"
}
```

## ***Grooming de las actividades de la etapa 3***

### Actividades planteadas en la estimación en la entrega 2:

- Desarrollo de vista en Angular para visualización de pedidos de colaboración cargados: 3 horas.
- Desarrollo de vista en Angular para establecer compromiso de ayuda con un pedido: 1 hora.
- Desarrollo de vista en Angular para marcar compromiso como cumplido: 1 hora.
- Desarrollo de lógica en backend para hacer consultas a la base de datos y obtener los pedidos de colaboración cargados: 2 horas.
- Desarrollo de lógica en backend para establecer compromiso de ayuda con un pedido de colaboración: 2 horas.
- Desarrollo de lógica en backend para marcar compromiso como cumplido: 1 hora.
- Implementar lógica de bonita para completar las nuevas tareas: 3 horas
- Creación de la API para visualización de pedidos de colaboración cargados, compromiso de ayuda con un determinado pedido y marcar compromiso como cumplido: 3 horas.
- Desarrollo del token en jwt para autenticación en la API: 2 horas.
- Ver documentación para subir al cloud: 1 hora
- Subir la API al cloud: 1 hora
- Consultas en clase: 20 minutos.
- Creación de documento Swagger para realizar pruebas: 20 minutos.
- Validación del grooming de la etapa 3: 20 minutos.



- Diseño de casos de prueba y testeo de los formularios, sus validaciones y persistencia en la base de datos: 1 hora.
- Estimación de las tareas de la etapa 4: 20 minutos

#### Actividades efectivamente realizadas:

- Desarrollo de lógica en backend para establecer compromiso de ayuda con un pedido de colaboración: **Estimación: 2 horas → Realidad: 2 horas.**
- Desarrollo de lógica en backend para marcar compromiso como cumplido: **Estimación: 1 hora → Realidad: 1 hora.**
- Creación de la API para visualización de pedidos de colaboración cargados, compromiso de ayuda con un determinado pedido y marcar compromiso como cumplido: **Estimación: 3 horas → Realidad: 3 horas.**
- Desarrollo del token en jwt para autenticación en la API: **Estimación: 2 horas → Realidad: 3 horas.**
- Ver documentación para subir al cloud: **Estimación: 1 hora → Realidad: 1 hora.**
- Subir la API al cloud: **Estimación: 1 hora → Realidad: 2 horas.**
- Consultas en clase: **Estimación: 20 minutos → Realidad: 40 minutos.**
- Creación de documento Swagger para realizar pruebas: **Estimación: 20 minutos → Realidad: 3 horas.**
- Validación del grooming de la etapa 3: **Estimación: 20 minutos → Realidad: 20 minutos.**
- Diseño de casos de prueba y testeo de los formularios, sus validaciones y persistencia en la base de datos: **Estimación: 1 hora → Realidad: 2 horas.**
- Estimación de las tareas de la etapa 4: **Estimación: 20 minutos → Realidad: 20 minutos.**

#### ***Estimaciones etapa 4***

- Relevamiento de actividades realizadas en etapas anteriores: 1 hora.
- Identificación y registro de desvíos respecto al plan original: 40 minutos.
- Definición de actividades pendientes hasta la entrega final: 1 hora.
- Armado del diagrama de Gantt en herramienta elegida: 3 horas.
- Revisión y ajuste de duraciones y dependencias en el Gantt: 40 minutos.
- Redacción de breve explicación del diagrama y justificación de desvíos: 1 hora.
- Revisión final del documento antes de la entrega: 30 minutos.
- Consultas en clase / equipo para validar el plan: 20 minutos.